Représentation d'informations - Les entiers

Introduction

Dans cette partie nous nous intéressons à la représentation d'informations dans un ordinateur. L'algèbre de Boole ne fournissant que 2 valeurs, il va être nécessaire d'utiliser des codes pour représenter des informations appartenant à des ensembles de plus de 2 valeurs. On parlera donc aussi bien de représentation que de codage.

Informations numériques

1. Dénombrement et comptage

On connaît plusieurs « catégories » de nombres qu'on peut caractériser par la taille de leur représentation :

Nombre de nombre infini, taille finie :
- entiers naturels,
- entiers relatifs,
- décimaux,

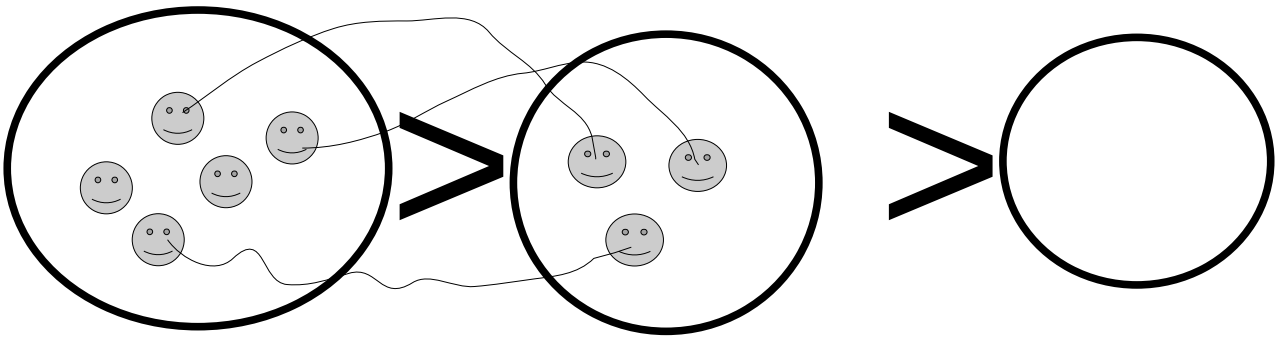
Nombre de nombres infini, taille infinie mais pouvant être représenté de manière finie par une fraction :
- rationnels,

Nombre de nombre infini, taille infinie :
- réels,
- complexes.

1 Commençons par le plus facile : les entiers naturels

Dans Entier Naturel il y a :
- Entier : nombre exact de « trucs ».
- Naturel : des « trucs » qui existent dans la nature.
- On peut ajouter (sans entrer dans un grand débat) l'absence de « truc », c'est à dire 0 (zéro) !

La notion importante sous-jacente, c'est la notion de comptage ou de dénombrement, qui est une relation d'ordre, si on sait dire (ou dessiner plutôt !) :



on sait compter et donc on sait manipuler des entiers naturels !

2 Codage

La question du codage commence dès qu'on veut nommer les quantités plutôt que les dessiner :
- On peut décider de donner un nom différent à chaque quantité, ce qui nous oblige à prévoir un nombre infini de nom !

– On peut choisir de regrouper les quantités par paquets, nommer un certain nombre fini de paquets et accumuler les noms dans un codage :
  1, 2, 3, 4, 5, 6, 7, 8, 9 puis au-delà on va dire qu'on a un paquet.
  On continue et on arrive à 2 paquets, 3 paquets, etc.
  Puis on va dire qu'on a un paquet de paquets et etc.

3 Formalisation

Dans la représentation précédente, appelons b le nombre choisi pour dire qu'on a 1 paquet, le nombre de truc dans un paquet est donc < b. On a donc besoin de b – 1 symboles. Si on ajoute un symbole pour représenter l'absence (0), on a besoin de exactement b symboles pour représenter des quantités, ou des entiers naturels.
La valeur représentée par un seul symbole s'appelle un chiffre.

Exemple :
b = 10, les symboles 0, 1, 2, 3, 4, 5, 6, 7, 8, 9

« cinq cent soixante-trois moutons » va se coder (en choisissant d'écrire de la gauche vers la droite pour des niveaux de paquets décroissants) :

563 moutons

Ce qui signifie qu'on a 3 moutons et 6 paquets de moutons et 5 paquets de paquets de moutons !

Formalisation de l'écriture :
1 à droite veut dire un !
1 en nombre de paquets veut dire dix = b
1 en nombre de paquets de paquets veut dire cent = b^2
….

On arrête de jouer :

$$563 = 5*10^2 + 6*10 + 3 = 5*10^2 + 6*10^1 + 3*10^0$$

Cas général :

$$N = \sum_{i=0}^{k-1} r_i * b^i$$

$avec$
N : $Nombre(de\,moutons)$
b : $base\,du\,codage$
k : $longueur\,de\,l'\,écriture\,du\,nombre$
r_i : $i^e\,symbole$

$$563 = \sum_{i=0}^{2} r_i * 10^i$$

$avec$
$r_0 = 3$
$r_1 = 6$
$r_2 = 5$

Pour ne pas qu'il y ait d'ambiguïté sur la base on peut la noter en indice : $(563)_{10}$

Exemple :
Passer en base 10 les nombres :
$(563)_8$
$(563)_{16}$
$(563)_7$
$(563)_2$ Attention pas possible ! Les chiffres doivent être inférieurs à la base.
$(10101101)_2$

Le problème restant consiste juste à choisir des symboles. Par exemple :

base 10 : 0, 1, 2, 3, 4, 5, 6, 7, 8, 9
base 2 : 0, 1
base 5 : 0, 1, 2, 3, 4 ou 0, 2, 4, 6, 8
base 4 : Ga, Bu, Zo, Meu
base 12 : 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, ∂, ∟
base 16 : 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F

Passer en base 10 les nombres :
$(3\partial 7 \llcorner)_{12}$
$(FFFF)_{16}$

2. Formalisation du changement de base

1 Rappel sur la division euclidienne

Calculer « à la main » 585 / 7

$\widehat{58}\ 5 \quad 5 < 7\, alors\ je\ prends\, 58$	7
$56 \quad = 8 * \mathbf{7}$	$8 \quad car\, 7 * 8 \leq 58\, et\, 7 * 9 > 58$
$2\ 5 \quad 58 - 56 = 2\, et\ je\ descend\, 5$	
$\widehat{25} \quad 2 < 7\, alors\ je\ prends\ 25$	$(8)\ 3 \quad car\, 7 * 3 \leq 25\, et\, 7 * \ > 21$
$21 \quad = 3 * \mathbf{7}$	
$4 \quad = 25 - 21$	
$4 < 7 \quad la\ division\ est\ terminée$	
$4 \quad reste\ de\ la\ division$	$83 \quad quotient\ de\ la\ division$

Conclusion :
$585 = 7 * 83 + 4$

La division euclidienne est l'opération qui à partir d'un nombre N et d'un diviseur d détermine le quotient q et le reste r tels que :
$N = d * q + r \quad avec \quad r < d$

2 Décomposition d'un nombre

Le but est de trouver l'écriture d'une quantité dans une base b.

En dépliant la formule :

$$N = \sum_{i=0}^{k-1} r_i * b^i$$
$$N = r_0 * b^0 + r_1 * b^1 + r_2 * b^2 + r_3 * b^3 + \ldots$$
$$N = r_0 + b*(r_1 + r_2 * b + r_3 * b^2 + \ldots)$$
$$N = r_0 + b*(r_1 + b*(r_2 + b*(r_3 + \ldots)))$$
$$avec\ r_0\ 1^{er}\ chiffre\ (à\ droite)\ du\ nombre\ N\ dans\ la\ base\ b$$
$$r_0 < b$$

En identifiant avec la division euclidienne on obtient :

$$N = b * q + r_0$$

et

$$q = r_1 + b*(r_2 + b*(r_3 + b*(r_4 + \ldots)))$$
$$avec\ r_1\ 2^{e}\ chiffre\ (à\ droite)\ du\ nombre\ N\ dans\ la\ base\ b$$
$$r_1 < b$$

etc.

Conclusion
par divisions euclidiennes successives on extrait les chiffres de la représentation en base b d'un nombre.

Exercice : écrire

$(263)_{10}$ en base 8, 16, 7, 2….
$(444)_{10}$ en base 16.

Correspondance des chiffres entre certaines bases

Remplir le tableau

Base 16	Base 8	Base 2		Base 10
0	00	0000		0
1	01	0001		1
2	02	0010		2
3	03	0011		3
4	04	0100		4
5	05	0101		5
6	06	0110		6
7	07	0111		7
8	10	1000		8
9	11	1001		9
A	12	1010		10

B	13	1011		11
C	14	1100		12
D	15	1101		13
E	16	1110		14
F	17	1111		15

Que constate-t-on ?

- il faut exactement 3 chiffres en base 2 pour représenter 1 chiffre en base 8,
- il faut exactement 4 chiffres en base 2 pour représenter 1 chiffre en base 16.

3. Conclusion sur le codage des nombres

La base choisie est une convention, décidée en général en fonction de contraintes pratiques :

- dix doigts → base 10 dans la vie de tous les jours,
- douze phalanges → base 12 (système duodécimal),
- quatre mots de vocabulaire → base 4 (shadok : Ga, Bu, Zo, Meu),
- **deux valeurs binaires → base 2 pour un ordinateur**.

4. Opérations sur les entiers

1 Addition

Rappel en base 10 :

```
    1 1 1
    1 2 3 4 5
  + 6 7 8 9 0
  ------------
    8 0 2 3 5
```

L'algorithme de l'addition de 2 nombres par cette méthode repose sur :

- 2 opérations élémentaires qui n'en sont qu'une :
  - l'addition de 2 chiffres,
  - l'addition de 2 chiffres plus une retenue valant 0 ou 1,
- l'algorithme de propagation de la retenue éventuelle de la droite vers la gauche.

L'addition de deux chiffres repose sur la table d'addition suivante :

+	0	1	2	3	4	5	6	7	8	9
0	0	1	2	3	4	5	6	7	8	9
1		2	3	4	5	6	7	8	9	0
2			4	5	6	7	8	9	0	1
3				6	7	8	9	0	1	2
4					8	9	0	1	2	3
5						0	1	2	3	4
6							2	3	4	5
7								4	5	6
8									6	7
9										8

Les cases en noir indiquent les cas où il y a retenue.

2 Exercice – addition en base quelconque

Calculer

$$(123)_5 + (404)_5$$

$$(10110111)_2 + (11010011)_2$$

3 Circuit réalisant l'addition binaire

En base 2 on peut utiliser la même méthode. La table d'addition est beaucoup plus simple puisqu'il n'y a que 2 chiffres (0 et 1) :

+	0	1
0	0	1
1	1	0

On voit que l'addition de 2 chiffres binaires A et B génère deux résultats :

- la somme S,
- la retenue R.

Ces deux résultats peuvent être vus comme des fonctions logiques des 2 chiffres additionnés.
Les tables de vérité de ces 2 fonctions sont :

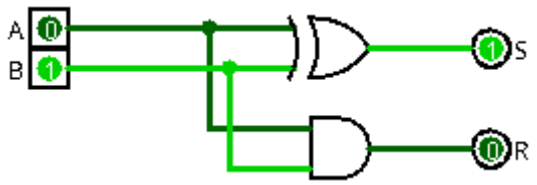
A	B	S	R
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

on en déduit les fonctions :

$$S = A \oplus B$$

$$R = A.B$$

et le circuit :



Pour additionner 2 nombres, il suffit d'intégrer l'algorithme de propagation de la retenue de la droite vers la gauche, cela se résume à la somme de 3 chiffres : le i^e^ chiffre de A : A_i, le i^e^ chiffre de B : B_i et la retenue précédente R_{i-1} et à la production de 2 résultats, la somme S_i et la retenue suivante R_i. La 1re retenue vaudra toujours 0.

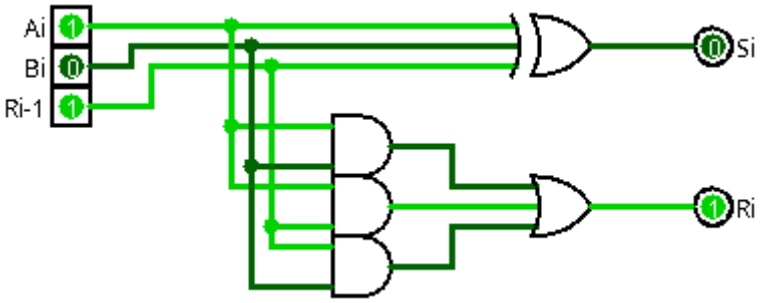
A_i	B_i	R_{i-1}		S_i	R_i
0	0	0		0	0
0	0	1		1	0
0	1	0		1	0
0	1	1		0	1
1	0	0		1	0
1	0	1		0	1
1	1	0		0	1
1	1	1		1	1

On déduit les fonctions :

$$S_i = A_i \oplus B_i \oplus R_{i-1}$$

$$R_i = (A_i . B_i) + (A_i . R_{i-1}) + (B_i . R_{i-1})$$

Et le circuit :



5. Codage des entiers relatifs dans un ordinateur – Le cas du '-'

Comment code-t-on les valeurs entières négatives en base 10 (dans la vie de tous les jours) ?

Une valeur négative est codée en utilisant sa valeur absolue et en mettant un « - » devant !

$$-3 = - |-3|$$

On pourrait faire pareil en base 2 : comment coder en binaire $(-13)_{10}$?

On commence par coder sa valeur absolue :
$(13)_{10} = (1101)_2$

Donc $(-13)_{10} = (-1101)_2$

Mais comment représenter le '-' sachant qu'on a que 2 symboles 0 et 1 ?

6. Proposition – mais c'est pas la bonne !

On utilise 0 et 1 pour coder le signe : 0 pour '+' et 1 pour '-', en décidant que le premier symbole d'un nombre binaire correspond au signe :

$$(-13)_{10} = (-1101)_2 = (11101)_2$$

Mais alors, comment différencier

$(-13)_{10} = (11101)_2$
de
$(+27)_{10} = (11011)_2$

Décider que le premier chiffre correspond au signe n'est pas suffisant comme contrainte.

On ajoute alors que les nombres seront toujours représentés avec un nombre de chiffres fixé, ce qui n'est pas une trop grande contrainte quand il s'agit de ranger ces nombres dans une mémoire d'ordinateur qui est déjà organisée avec des contraintes de taille.

Exercice :
on décide de coder les entiers relatifs sur 3 chiffres, donner toutes les valeurs possibles.

Base 2		Base 10
0000		0
0001		1
0010		2
0011		3
0100		4
0101		5
0110		6
0111		7
1000		-0 ?
1001		-1
1010		-2
1011		-3
1100		-4
1101		-5
1110		-6
1111		-7

Défaut de ce codage
si on calcule 5 + (-5)

```
  0 1 0 1
+ 1 1 0 1
---------
  0 0 1 0
```

On ne trouve pas 0 mais 2 !

Ce codage ne respecte pas l'addition !

7. Opération de soustraction

Rappel en base 10 :

Effectuer la soustraction en montrant progressivement les colonnes de la gauche vers la droite.

D'abord une sans retenue :

```
  2 9 5
- 1 3 4
-------
  1 6 1
```

Puis une avec retenue sans problème toujours en masquant :

```
  2 14 7
```

```
 - 1  6 6
   1
 ----------
       8 1
```

Dans le cas où le chiffre du haut est inférieur au chiffre du bas, on prend une unité dans la colonne de gauche qu'on ajoute, et on la « rend » en la soustrayant ensuite. On calcule donc successivement :

- 7 – 6 = 1,
- 14 – 6 = 8,
- 2 – 1 – 1.

Algorithme de la soustraction :

l'algorithme de la soustraction repose sur :

- la soustraction de 2 chiffres que l'on peut résumer par une table comme pour l'addition,
- la soustraction de deux chiffres et la soustraction de 1,
- la propagation éventuelle d'une retenue de la droite vers la gauche.

Une qui pose problème :

```
   ...  10  10  11  12  9  14
 -              3   4   8  6
   ...  1   1   1       1
 ----------------------------
   ...  9   9   7   8   0  8
```

Quel est le problème ?

Le problème est que l'algorithme de la soustraction précédent n'est pas complet, il faut ajouter une première opération qui consiste en la comparaison des valeurs absolues des deux nombres, de manière à mettre « en haut » le plus grand et en bas le plus petit, et à retenir que si le plus grand était le nombre à soustraire, il faudra remettre le résultat en négatif :

$$A - B = \begin{cases} A - B & si \ |A| > |B| \\ -|B - A| & si \ |A| < |B| \end{cases}$$

Sur un exemple plus petit :

35 - 43

en s'arrêtant à 3 chiffres

```
       3 5
 -     4 3
   1
 ----------
   9 9 2
```

Et la soustraction dans le « bon » sens :

```
       4 3
```

-	3	5
		8

On peut remarquer que :
992 + 8 = 1000
et si on ne conserve que 3 chiffres (de gauche) on trouve que :
992 + 8 = 0.

Dans ce codage sur 3 chiffres 992 est bien l'opposé de 8 au sens de l'addition que l'on note habituellement -8 :
Cette façon de coder une valeur négative s'appelle codage en complément à la base dans le cas général, et codage en complément à 10 dans ce cas.

En résumé on a trouvé une deuxième façon de coder les entiers relatifs en base 10 :
1) codage « habituel » :
- les valeurs positives sont codées par la formule polynomiale

$$N = \sum_{i=0}^{k-1} r_i * b^i$$

$avec$
N : $Nombre(de\,moutons)$
b : $base\,du\,codage$
k : $longueur\,de\,l'\,écriture\,du\,nombre$
r_i : $i^e\,symbole$

- les valeurs négatives sont codées par leur valeur absolue précédée d'un symbole '-'

2) codage en complément à dix :
- les valeurs positives sont codées par la formule polynomiale
- les valeurs négatives sont codées en complément à 10

Cette deuxième façon de coder a un inconvénient, c'est qu'il faut imposer une taille maximale pour la représentation des entiers.

8. Autre proposition pour le codage des valeurs négatives

Nous allons étudier ce codage en complément à la base sur les nombres codés en base 2, c'est à dire le codage en complément à 2.
Tout d'abord, dans le cas qui nous intéresse de représentation de valeurs dans un ordinateur, la contrainte sur la taille des nombres n'est pas très importante car nous l'avions déjà sur la proposition précédente.

Comment définir le codage d'une valeur négative en complément à 2 ?
Pour une taille donnée (un nombre de chiffres), si N est une valeur positive codée en base 2, le codage de -N est tel que N + (-N) = 0.
Par exemple, sur 4 bits, quel est le codage de -4 ?

4 en base 2 sur 4 bits s'écrit : 0100
- 4 en complément à 2 sur 4 bits est la valeur composée de 4 chiffres binaires $b_3 b_2 b_1 b_0$ telle que :

	0	1	0	0
+	b_3	b_2	b_1	b_0
~~1~~	0	0	0	0

Reste à résoudre 4 équations pour trouver b_3, b_2, b_1, b_0 :
attention il s'agit bien d'addition et pas de OU !

$0 + b_0 = 0$ qui donne $b_0 = 0$
$0 + b_1 = 0$ qui donne $b_1 = 0$
$1 + b_2 = 0$ qui donne $b_2 = 1 \, avec \, retenue$
$1 + 0 + b_3 = 0$ qui donne $b_0 = 1 \, avec \, retenue$ attention il ne faut pas oublier la retenue précédente.
Il reste une retenue qui n'est pas conservée puisqu'on n'utilise que 4 chiffres.

$(-4)_{10} = (1100)_2$ en complément à 2

Exercice :
Remplir le tableau suivant

Base 2 – complément à 2	Base 10
0000	0
0001	1
0010	2
0011	3
0100	4
0101	5
0110	6
0111	7
1000	? puis -8
1001	-7
1010	-6
1011	-5
1100	-4
1101	-3
1110	-2
1111	-1

On peut vérifier que l'addition d'un nombre avec son opposé donne bien 0.

Cas de 1000 : quelle valeur en base 10 est représentée ?

calculons :

	1	0	0	0	x
+	0	1	1	0	6
	1	1	1	0	-2

	1	0	0	0	x
+	0	0	0	1	1
	1	0	0	1	-7

Des additions précédentes on peut déduire x = - 8, ce qui nous donne une valeur de plus.

9. Intérêt du codage en complément à 2

1 Vérification de l'opposé

Calcul de 5 + (-5) :

```
  0 1 0 1
+ 1 0 1 1
---------
  0 0 0 0
```

Calcul de 2 + (-2) :

```
  0 0 1 0
+ 1 1 1 0
---------
  0 0 0 0
```

Ce codage respecte l'addition pour la propriété de symétrie (tout nombre possède un symétrique appelé opposé pour l'addition, tel que la somme du nombre et de son opposé est nulle).

2 Vérification de l'addition :

Calcul de (-3) + (-4)

```
  1 1 0 1
+ 1 1 0 0
---------
  1 0 0 1
```

On trouve bien -7

Calcul de (-5) + (2)

```
  1 0 1 1
+ 0 0 1 0
---------
  1 1 0 1
```

On trouve -3

Calcul de (-4) + (6)

$$\begin{array}{r} 1\,1\,0\,0 \\ +\,0\,1\,1\,0 \\ \hline 0\,0\,1\,0 \end{array}$$

On trouve 2

Ce codage respecte l'addition et donc la soustraction peut être réalisée facilement : A – B = A + (-B)

3 Méthode de codage en complément à deux

Pour passer d'un nombre à son complément à 2, il suffit de transformer le nombre en son complément à 1, qui s'obtient en inversant tous les bits du nombre, puis d'ajouter 1.

Passer en complément à 2 les nombres suivants
sur 4 bits : 3, 5, 6, 0, -4
sur 8 bits : -92, 47, -153

4 Capacité de codage

Combien de valeurs différentes (quelles sont les bornes minimales et maximales) peut on coder en complément à 2 sur 4 bits, 8 bits, 16 bits, etc.

Remarque sur le stockage des valeurs

Le stockage d'informations dans un ordinateur se fait dans des circuits (composants) électroniques nommés mémoires (!). Le nombre d'informations que l'on peut stocker est défini par la quantité de circuits de mémoire installés dans l'ordinateur.
L'unité de base de stockage est bien sûr le chiffre binaire que l'on nomme bit de la contraction de l'anglais **b**inary dig**it**. Avec un bit on ne peut stocker que 2 valeurs différentes, représentées par les 2 valeurs binaires 0 et 1, ce qui est peu, aussi la mémoire est organisée par paquets de plusieurs bits qui peuvent être accédés ensemble. La taille de ces paquets est très généralement une puissance de 2, ce qui est pratique et cohérent avec la manipulation de valeurs binaires.
Le premier niveau de regroupement est de 8 bits, qu'on appelle octet en français et byte en anglais (attention donc à la prononciation pour ne pas confondre bit et byte).

Question : de combien de chiffres hexadécimaux a-t-on besoin pour coder un octet ?

Parenthèse sur la quantité de mémoire (histoire de parler de puissance de 2 !) :
cf. http://fr.wikipedia.org/wiki/Octet

Quand on parle de taille mémoire pour un ordinateur on la mesure en nombres d'octets, et ces nombres étant couramment très grands on utilise, comme pour toute unité, des multiples. Pour des raisons historiques ces multiples existent sous deux formes :

- on a commencé par utiliser des multiplicateurs puissances de 2 ce qui a donné :
  - multiplicateur de l'ordre de 1000 : $2^{10}=1024$ qui s'appelait kilo-octet
  - multiplicateur de l'ordre de 1 million : $2^{20}=1048576$ qui s'appelait méga-octet
  - multiplicateur de l'ordre de 1 milliard : 2^{30} qui s'appelait giga-octet
  - etc.
- puis les multiplicateurs ont été normalisés et alignés sur les multiples standards et les anciens ont été renommés :

Préfixes normalisés	Préfixes binaires
10^3 kilooctet	2^{10} kibioctet
10^6 mégaoctet	2^{20} mébioctet
10^9 gigaoctet	2^{30} gibioctet
10^{12} téraoctet	2^{40} tébioctet
10^{15} pétaoctet	2^{50} pébioctet
10^{18} exaoctet	2^{60} exbioctet
10^{21} zéttaoctet	2^{70} zébioctet
10^{24} yottaoctet	2^{80} yobioctet

Quelques quantités :
Combien de valeurs différentes peut-on stocker avec 1 octet :
Avec 1 octet (8 bits) on écrit des valeurs binaires de 00000000 à 11111111, cela représente 256 codes différents. On peut donc représenter 256 informations différentes sur 1 octet.

Avec 2 octets – 16 bits, on représente 65536 valeurs différentes.

On utilise par exemple 3 octets pour coder des couleurs dans une image projetée en synthèse additive :
les couleurs sont représentées comme l'addition de 3 couleurs primaires Rouge, Vert, Bleu, l'intensité de chacune des 3 couleurs étant codée sur un octet

En pratique les nombres seront codés sur des paquets de taille multiple de 8 bits.

Exemple pour 8 :

$$(-13)_{10} = (10001101)_2$$

et

$$(+27)_{10} = (00011101)_2$$